

AML

Davide Volpi

January 5, 2026

Contents

1	General Concepts and Notations	3
1.1	Tasks	3
1.2	Successfulness of a Model	5
1.3	Methodological Practices for the Learning Procedure	5
1.4	Ill-Posed Problem	5
1.5	Hypothesis Spaces	6
2	Bayesian Methods for ML	7
2.1	Introduction	7
2.1.1	Procedure	7
2.1.2	Obtention of θ	7
2.2	Bayesian (Linear) Regression	8
3	Generalized Linear Models - GLMs	9
4	Generative Classifiers (Derived from Bayes Formula)	10
4.1	K-Nearest Neighbours (KNN)	11
5	Perceptron (Rosenblatt 58)	12
5.1	Dual Perceptron	12
5.2	The Delta Rule	13
6	Layer Types: UA Property	15
7	VC-Dimension Theory (Binary Classifications)	16
7.1	Application to SVM	16
7.1.1	Quasi Linsep Case	17
8	Learning in Functional Space	18
8.1	Kernel Properties	19
8.2	Kernel Functions for Non-Vectorial Data	20
8.3	Polynomial Kernel in SVM vs Linear Regression	20
8.4	Design of (Metric) Distances and Similarities	20
9	Hopfield Networks	22
9.1	Introduction and Core Concepts	22
9.2	Architecture and State	22
9.3	Formal Analysis and Dynamics	22
9.3.1	Neuron Update Rule	22
9.3.2	Update Modes	22
9.4	Hebbian Learning (Storing Memories)	22
9.4.1	Single Pattern Case ($l = 1$)	22
9.4.2	Multiple Patterns Case ($l \geq 1$)	23
9.5	Stability Analysis	23
9.6	Capacity	23
9.7	Spurious States	23

9.8	The Energy Function	23
9.9	Applications	24
9.9.1	Optimization Problems	24
9.9.2	Pattern Reconstruction	24
10	Neural Networks Training $\rightarrow$ Kernel Functions	25
10.1	Kernel Functions $\rightarrow$ Neural Networks Training	25
11	Graph Neural Networks	26
12	Exam 2024	27
13	Exam 2025	28
14	Mock Test	30

1 General Concepts and Notations

Let $f(x; \theta)$ or $f_\theta(x)$ denote a function that depends on the data x and the parameters θ .

Hypothesis Space: linear function $\mathcal{F} := \{f_\theta : \mathcal{X} \rightarrow \mathcal{Y} \mid \theta \in \mathbb{R}^\ell\}$ where $\mathcal{X}$ is the data/input space and $\mathcal{Y}$ is the target space.

Hypothesis Space for Polynomials: Let $P_M := \{x \mapsto c_0 + \sum_{j=1}^M c_j x^j, c_M \neq 0\}$ be the set of all polynomials of degree M . Then $\mathcal{F} := \cup_{M \geq 0} P_M$ is the required space.

Model: any member of $f \in \mathcal{F}$.

We assume that there is a data generation mechanism in the form of a joint probability distribution $\rho(x, y)$ and a data sample $D_n \stackrel{iid}{\sim} \rho^n$, $D_n = \{(x^1, y^1), \dots, (x^n, y^n)\}$.

Loss Function: $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^+ \cup \{0\}$ and we use it as $\ell(y, \hat{y})$. Its properties are:

- $\ell(y, \hat{y}) \geq 0$
- $\ell(y, \hat{y}) = 0 \Leftrightarrow y = \hat{y}$
- $\ell(y, \hat{y})$ increases monotonically with $|y - \hat{y}|$

Regularizer: $\Omega : \mathcal{F} \rightarrow \mathbb{R}^+ \cup \{0\}$, where Ω measures the magnitude of f ($\Omega(f) = \frac{1}{q} \|f\|_q^q$).

Risk of f : $R[f] = \mathbb{E}_{(x,y) \sim \rho} [\ell(y, f(x))]$.

Empirical Risk of f on D_n : $\hat{R}_{D_n}[f] = \frac{1}{n} \sum_{i=1}^n \ell(y^i, f(x^i))$.

Machine Learning Algorithm: takes a hypothesis space $\mathcal{F}$, a sample D_n , a loss ℓ , a regularizer Ω , and produces a model with an estimate of its generalization error.

Proposition: For fixed f_θ , the expected empirical risk equals the true risk:

$$\mathbb{E}_{D_n \stackrel{iid}{\sim} \rho^n} [\hat{R}_{D_n}[f_\theta]] = R[f_\theta].$$

Bias of f on x : $bias(f; x) := \mathbb{E}_{D_n}[f(x)] - f_{sq}^*(x) \rightarrow bias(f) := \int_{\mathbb{R}^d} bias(f, x) p(x) dx$.

Variance of f on x : $var(f; x) := \mathbb{E}_{D_n}[(f(x) - \mathbb{E}_{D_n}[f(x)])^2] \rightarrow var(f) := \int_{\mathbb{R}^d} var(f, x) p(x) dx$.

MSE of f on x : $MSE(f; x) := \mathbb{E}_{D_n}[(f(x) - f_{sq}^*(x))^2]$.

Noise of f in x : $\sigma^2(x) := \int_{\mathbb{R}} [y - f_{sq}^*(x)]^2 p(y|x) dy \rightarrow \sigma^2 := \int_{\mathbb{R}^d} \sigma^2(x) p(x) dx$

Standard Regression Theory: $y = f(x) + \varepsilon$, where ε is a random variable with:

- $\mathbb{E}(\varepsilon) = 0 \leftarrow \text{free}$
- $Var(\varepsilon) < \sigma^2 \leftarrow \neq \infty$
- ε, x are independent

Bias/Variance Decomposition: For every f the risk of x can be decomposed as

$$R[f] = (bias(f))^2 + Var(f) + \sigma^2 = MSE(f) + \sigma^2 \quad MSE(f, x) = [Bias(f, x)]^2 + Var(f, x)$$

1.1 Tasks

- **Regression:** we take $m = 1$. Under the assumption of Gaussian noise, by applying the MLE the obtained loss is the squared error:

$$\ell_{sq}(y, \hat{y}) = (y - \hat{y})^2$$

TH: Under ℓ_{sq} , if there are no restrictions in the choice of f , the minimization of the risk leads to

$$f_{sq}^*(x) := \int y p(y|x) dy = \mathbb{E}[Y|X = x] \quad (\text{statistical problem})$$

Optimisation Problem:

$$f^* = \arg \min_f \mathbb{E}[\ell_{sq}(Y, f(X))]$$

Existence/Uniqueness: a solution f^* always exists under finite variance of Y , is unique in the space of measurable functions and is given by the conditional expectation $f^*(x) = \mathbb{E}[Y|X = x]$.

Bayes Risk (lowest achievable error):

$$R[f_{sq}^*] = \mathbb{E}[(Y - f_{sq}^*(X))^2] \quad (\text{irreducible noise variance} \quad \mathbb{E}[\varepsilon^2])$$

Generalization Error: of an estimator $\hat{f}$ is the expected risk on unseen data:

$$R[\hat{f}] = \mathbb{E}[(Y - \hat{f}(X))^2] \quad (\text{cross validation})$$

Uncertainty:

- **Frequentist:** confidence intervals for the estimated parameters or prediction intervals for new data, obtained from the variance of the estimator.
 - **Bayesian:** a posterior distribution over the parameters or functions, giving a full probabilistic description of uncertainty.
 - **Prediction intervals:** intervals that combine both estimation uncertainty and irreducible noise variance.
- **Binary classification:** given $\mathcal{X}$ input space and $\mathcal{Y}$ output space $= \{-1, 1\}$. Let $p(+1|x), p(-1|x)$ are the (posterior) probabilities of x belonging to class $+1, -1$ respectively, with $\ell_{0|1}(y, \hat{y}) = \mathbb{1}_{y \neq \hat{y}}$.

Risk:

$$R[f] = \int_{\mathbb{R}^d} \sum_{y \in \{-1, 1\}} \mathbb{1}_{y \neq f(x)} p(y|x) p(x) dx$$

Bayes Classifier: The function that minimizes R is

$$f_{0|1}^*(x) = \begin{cases} +1, & \text{if } p(+1|x) > p(-1|x), \\ -1, & \text{otherwise.} \end{cases} \quad (\text{statistical problem})$$

Bayes Risk (lowest achievable error):

$$R[f_{0|1}^*] = \int_{\mathbb{R}^d} \min\{p(+1|x), p(-1|x)\} p(x) dx$$

Optimization problem: The Bayes classifier is also the solution to

$$f^* = \arg \min_f \mathbb{E}[\ell_{0|1}(Y, f(X))]$$

Existence/uniqueness: A solution always exists in the space of measurable functions. For 0-1 loss, it is unique almost everywhere (ties can occur but are rare).

Generalization error: For any classifier $\hat{f}$, the generalization error is

$$R[\hat{f}] = \mathbb{E}[\mathbb{1}_{Y \neq \hat{f}(X)}] \quad (\text{cross validation})$$

Uncertainty:

- **Probabilistic:** outputs $p(y|x)$ quantify confidence in predictions.
 - **Frequentist:** confidence intervals via bootstrap or parameter variance.
 - **Bayesian:** posterior distributions over model parameters.
 - **Margin-based:** distance from decision boundary.
- $c > 2$ **classes:** given $\mathcal{X}$ input space and $\mathcal{Y} = \{1, 2, \dots, \ell\}$, the risk is

$$R[f] = \int_{\mathbb{R}^d} \sum_{i=1}^{\ell} \mathbb{1}_{f(x) \neq c_i} p(c_i|x) p(x) dx$$

Bayes Classifier:

$$f^*(x) = \arg \max_{i=1, \dots, \ell} p(c_i|x)$$

Bayes Risk:

$$R[f^*] = \int_{\mathbb{R}^d} \min_{i=1, \dots, \ell} p(c_i|x) p(x) dx$$

All other points (optimization view, existence, generalization, uncertainty) extend directly from the binary case.

1.2 Successfulness of a Model

A model is successful when it has these properties:

- **Robustness** to outliers, errors and/or wrong model assumptions.
- **Stability** against variations of the training data.
- **Efficiency/Scalability** in the computational sense (handle large datasets).
- **Flexibility** to perform different tasks.
- **Complexity Control**, meaning complexity surplus and explicit complexity control.
- **Versatility** to accept different data types and incorporate prior knowledge.
- **Reproducibility** to be in accordance with the specific method.
- **Explainability** important in critical applications with human lives or sensitive information.

1.3 Methodological Practices for the Learning Procedure

- **Proper resampling:** recommended ($t \text{ times} \times k \text{ fold}$) - cross validation, proper use of test data and pay attention to methodological flaws or biases.
- **Sensible pre-processing:**
 - Treatment of lost values.
 - Treatment of anomalous values.
 - Treatment of incoherent or incorrect values.
 - Treatment of new/mixed data types.
 - Creation/extraction of new variables that can be useful.
 - Normalisation of variables and/or normalisation of data points.
 - Transformation of variables.
- **Feature manipulation:**
 - Elimination of irrelevant or less important variables.
 - Elimination of some redundant variables.
 - Treating variables differently.
- **Selection of hyperparameters:** use safe and efficient procedure and avoid exhaustive search.
- **Honest performance evaluation:** proper use of test data and give CIs.
- **Honest comparison to other MLAs or models:** use of statistical test.

1.4 Ill-Posed Problem

- The solution may not exist.
- The solution may not be unique.
- The solution does not depend in the initial conditions.

Regularization is how we fix the fact that generally learning from data is ill-posed.

1.5 Hypothesis Spaces

Formal definition of all linear binary classifiers (hyperplanes) in d variables:

$$Y = \{-1, 1\} \quad f_{\theta, b}(x) = \text{sign}(\theta^\top x + b) \quad \mathcal{H} = \{f_{\theta, b} : \mathbb{R}^d \rightarrow Y \mid f_{\theta, b} = \text{sign}(\theta^\top x + b), \theta \in \mathbb{R}^d, b \in \mathbb{R}\}$$

Formal definition of all one hidden-layer MLPs binary classifiers in d variables:

$$f(x) = \text{sign}\left(\sum_{j=1}^h v_j \sigma(\theta_j^\top x + b_j) + c\right) \quad \mathcal{H} = \{f : \mathbb{R}^d \rightarrow \{-1, 1\} \mid f(x) = \text{sign}\left(\sum_{j=1}^h v_j \sigma(\theta_j^\top x + b_j) + c\right), w_j \in \mathbb{R}^d\}$$

Formal definition of all the polynomials of a given maximum degree M :

$$P_M := \{x \mapsto c_0 + \sum_{j=1}^M c_j x^j, c_M \neq 0\} \text{ set of all polynomials of degree } M, \mathcal{F} := \cup_{M \geq 1} \mathcal{P}_M \text{ required space}$$

2 Bayesian Methods for ML

2.1 Introduction

We depart from an observed data sample $D^n := \{(x^1, y^1), \dots, (x^n, y^n)\}$, with $x^i \in \mathcal{X} = \mathbb{R}^d, y^i \in \mathcal{Y} = \mathbb{R}$, a loss function ℓ and a hypothesis space of parametrised functions (models):

$$\mathcal{F} := \{x \rightarrow f_\theta(x, \theta), x \in \mathbb{R}^d, \theta \in \mathbb{R}^s\}$$

We define the Bayes formula as:

$$p(\theta|D^n) = \frac{p(D^n|\theta)p(\theta)}{\int_{\mathbb{R}^s} d\theta' p(D^n|\theta')p(\theta')}$$

- The denominator is not needed since is a number.
- $p(\theta)$ is the **prior** (confidence in θ before observing D^n).
- $p(D^n|\theta)$ is the **likelihood** (probability of observing D^n if the parameters were θ)
- $p(\theta|D^n)$ is the **posterior** (confidence in θ after observing D^n).

The decision boundary between every pair of (distinct) classes is a hyperquadratic.
If we assume $\varepsilon_1 = \dots = \varepsilon_c = \varepsilon$ the decision boundaries are hyperplanes.

2.1.1 Procedure

1. Define the functional form off the possible models (elements of $\mathcal{F}$), the prior distribution $p(\theta)$ and obtain the data sample D^n .
2. Convert the model into a parameterised probability distribution. Generalize it to the full likelihood function on D^n :

$$\mathcal{L}(\theta) := p(D^n|\theta) = \prod_{i=1}^n p(y^i|x^i, \theta) \quad or \quad -\mathcal{L}(\theta) = -\sum_{i=1}^n \ln p(y^i|x^i, \theta)$$

3. Obtain the posterior distribution:

$$p(\theta|D^n) = \frac{p(\theta) \prod_{i=1}^n p(y^i|x^i, \theta)}{\int_{\mathbb{R}^s} d\theta' p(\theta') \prod_{i=1}^n p(y^i|x^i, \theta')}$$

4. Decide a method to make predictions when new x data arrive.

2.1.2 Obtention of θ

Once we have the posterior, we can proceed in several ways:

- Eliminate (or ignore) the prior $p(\theta)$ on θ (not being Bayesian). The problem then becomes:

$$\hat{\theta}_{ML} := \arg \max_{\theta} p(D^n|\theta)$$

- Calculate the most probable value of the parameter vector:

$$\hat{\theta}_{ML} := \arg \max_{\theta} p(\theta|D^n) = \arg \max_{\theta} p(D^n|\theta)p(\theta)$$

This is called **maximum a posteriori** or **MAP estimator**.

- Calculate the average parameter vector:

$$\hat{\theta}_{AVG} := \int_{\mathbb{R}^s} d\theta p(\theta|D^n)p(\theta)$$

2.2 Bayesian (Linear) Regression

Let us define a class of (linear) models for regression:

$$\mathcal{F}_{lin} := \{x \rightarrow w^\top x + w_0; x, w \in \mathbb{R}^d, w_0 \in \mathbb{R}\}$$

We then consider an adequate set of basis functions of BFs $\phi_1, \dots, \phi_D : \mathbb{R}^d \rightarrow \mathbb{R}$, with $\phi_1(x) := 1$ to include w_0 . We group the BFs into ϕ and redefine the class to accomodate the new parameter vector:

$$\mathcal{F}_{lin} := \{x \rightarrow v^\top \phi(x); x \in \mathbb{R}^d, v \in \mathbb{R}^D\}$$

- We write D^n as if split into $\{X, y\}$, being $X_{n \times d}$ the matrix whose i -th row is $(x^i)^\top$ and $y = (y_1, \dots, y_n)^\top$ is the vector of targets.
- We write D^n as if split into $\{\Phi, y\}$, with $\Phi_{n \times D}$, where $\Phi_{ij} = \phi_j(x^i)$ (design matrix).

Then we proceed as follow:

3. For an arbitrary $x \in \mathbb{R}^d$, we decide that a suitable pdf for the conditional distribution of the y is that of a univariate Gaussian, that is, $p(y|\phi(x), v, \sigma^2) = \mathcal{N}(y; f_v(x), \sigma^2)$, where $f_v(x) = v^\top \phi(x) \in \mathcal{F}_{lin}$. The prior pdf for the v parameters is taken to be a multivariate Gaussian $p(v) = \mathcal{N}(v; 0, \sigma_v^2 I)$. We then obtain the data D^n and our $D + 2$ model parameters are $\theta(v_1, \dots, v_D, \sigma^2, \sigma_v^2)$. The prior is:

$$p(v) = (2\pi\sigma_v^2)^{-\frac{D}{2}} \exp\left(-\frac{v^\top v}{2\sigma_v^2}\right)$$

4. The log-likelihood function is

$$\sum_{i=1}^n \ln\{(2\pi\sigma^2)^{-\frac{1}{2}} \exp\left(-\frac{1}{2\sigma^2}\{y^i - f_v(x^i)\}^2\right)\}$$

Therefore (minus) the numerator in the Bayes formula will be:

$$\frac{n}{2} \ln(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^n \{y^i - f_v(x^i)\}^2 + \frac{D}{2} \ln(2\pi\sigma_v^2) + \frac{1}{2\sigma_v^2} v^\top v$$

From the point of view of v the first and third terms are constants; we can calculate the maximum a posteriori or MAP estimator by solving the optimization problem:

$$\hat{v}_{MAP} := \arg \min_{v \in \mathbb{R}^D} \left\{ \frac{1}{2} \|y - f_v(X)\|^2 + \frac{\lambda}{2} \|v\|^2 \right\}, \lambda = \frac{\sigma^2}{\sigma_v^2} > 0$$

This problem is recognized to be analogous to **ridge regression**.

The Bayesian setting gives us the theoretically optimal value for the regularization hyperparameter $\hat{\lambda} := \frac{\sigma^2}{n\sigma_v^2}$, under the chosen Gaussianity assumption. The value is the best possible and that it actually depends on the available dataset D^n .

Now that we can calculate the **posterior distribution**, a convolution of two Gaussians and therefore another Gaussian, as $p(v) = p(v|D^n) = \mathcal{N}(v; \mu_n, \Sigma_n)$, where $\mu_n := \sigma^2 \Sigma_n \Phi^\top y$ and $\Sigma_n := (\sigma_v^2 I + \sigma^{-2} \Phi^\top \Phi)^{-1}$.

The **full Bayesian approach** involves the obtention of the **posterior predictive distribution**:

$$p(y^{new}|x^{new}, D^n, \sigma_v^2, \sigma^2) = \int dv p(y|\phi(x), v, \sigma^2) p(v|D^n, \sigma_v^2, \sigma^2)$$

where x^{new} is the new point of interest. Again this integral involves the convolution of two Gaussian distributions, and can be expressed algebraically:

$$p(y^{new}|x^{new}, D^n, \sigma_v^2, \sigma^2) = \mathcal{N}(y^{new}; \mu_n^\top x^{new}, \sigma^2(x^{new}))$$

being $\sigma^2(\cdot)$ the variance function of the predictive distribution

$$\sigma^2(x) = \sigma^2 + \phi(x)^\top \Sigma \phi(x)$$

The first term represents the noise in the data whereas the second term reflects the uncertainty associated with the v parameters. It can be shown that, as $n \rightarrow \infty$, the second term in the variance tends to zero, thus $\sigma^2(x)$ tends to σ^2 .

3 Generalized Linear Models - GLMs

The target variable types, specially useful in ML:

Target Type	Name of Algorithm (linear)
continuous	linear regression
binary class	logistic regression
multiclass	multinomial regression
ordinal	ordinal regression
counter	Poisson regression
continuous (positive)	?
time to failure	survival time regression

Linear models are functions of the form:

$$\mu_i = \mathbb{E}(Y_i|X = x_i) = w \cdot x^i + w_0$$

or, if you want to introduce Bayes functions where $\phi_\gamma : \mathbb{R}^d \rightarrow \mathbb{R}$ we have D Bayes functions $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$. Given $\phi_1(x) = 1, z^i := \phi(x^i)$ and we obtain

$$\mu_i = \mathbb{E}(Y_i|Z = z_i) = w \cdot z^i$$

In the standard linear regression settings $y_i = f(z^i; w) + \varepsilon_i$, the typical choice of the distribution of ε is

$$\varepsilon_i \stackrel{iid}{\sim} \mathcal{N}(0, \sigma^2) \rightarrow Y_i \stackrel{iid}{\sim} \mathcal{N}(y_i; f(z^i; w), \sigma^2)$$

In many cases, we need a different distribution than the Gaussian and in many cases we may need a more general relationship between the $\mathbb{E}$ and the linear part:

- There exists a **exponential family of distribution** including: $\gamma, \beta, \mathcal{N}, inv - \mathcal{N}, \mathcal{X}^2, Poisson, \dots$

$$f(y; \theta) = s(y)t(\theta)e^{a(y)b(\theta)} \quad \text{with } a, b, s, t \text{ known functions}$$

We are going to accept as "noise" any distribution belonging to this family.

- We add an interface function h , called **link function**, to the left part:

$$\mu_i = h(\mathbb{E}(Y_i|X = x_i)) = w \cdot x^i + w_0$$

GLMs can handle natively categorical values.

The procedure for fitting a GLM to data is:

1. Gather a data sample $D_n := \{(x^1, y^1), \dots, (x^n, y^n)\}, (x^i, y^i) \stackrel{iid}{\sim} f$.
2. Decide the type of the response (target) variable.
3. Decide an appropriate conditional distribution $P_{Y|X} \rightarrow f_{X,Y}(x, y) = P_{X|Y}(y|x)p_X(x)$ as a function the target type. A **discriminative** method focuses on the conditional distribution and ignores the other ones. Creates a function f to model the direct relation $X \rightarrow Y$ (GLMs are discriminative). A **generative** method does consider the input distribution.
4. Build the likelihood function $\alpha(\mu) = \prod_{i=1}^n P_{Y|X}(y^i; f(x; w^i))$ and $\min\{-\log \alpha\}$ to obtain the error.

$$f(x^i; w) = \mu_i = h(\mathbb{E}(Y_i|X = x_i)) = w \cdot x^i + w_0$$

5. Now we have $\sum_{i=1}^n \ell(y^i, f(x^i)) \min(w, w_0)$. Typically GLM are solved using Newton-Rapson:

$$w(t+1) = w(t) - H(w(t))^{-1} \cdot \nabla(w(t)) \quad \text{repeated until convergence}$$

Almost always, converges in less than 10 iterations.

Examples:

- For the linear regression $h(z) = z$.
- For the logistic regression $h(z) = \text{logit}(z) = \ln(\frac{z}{1-z})$. The inverse of the logit is the logistic.

4 Generative Classifiers (Derived from Bayes Formula)

In a ML context (classification), we have a set of classes $c \geq 2, \mathcal{C} = \{c_1, \dots, c_c\}$, data $x \in \mathbb{R}^d$:

$$P(c_j|x) = \frac{p(x|c_j) \cdot P(c_j)}{\sum_{l=1}^c p(x|c_l) \cdot P(c_l)}, j \in \mathcal{C}$$

Bayes (optimal) classifier: the prediction of x in class j if

$$j = \arg \max_{l \in \mathcal{C}} P(c_l|x)$$

The error committed by this decision is:

$$\sum_{l \in \mathcal{C} \setminus \{j\}} P(c_l|x) = 1 - P(c_j|x)$$

Reminder of a pdf of a random variable that is normally distributed:

$$x \in \mathbb{R}^d, p(x) = ((2\pi)^{d/2})^{-1} \exp(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)) \sim \mathcal{N}(x; \mu, \Sigma)$$

Let's assume that $X|_{class j} \sim \mathcal{N}(x; \mu_j, \Sigma_j)$

$$\arg \max_{j \in \mathcal{C}} \{P(x|c_j)P(c_j)\} = \arg \max_{j \in \mathcal{C}} \{h(P(x|c_j) \cdot P(c_j))\} \quad h \text{ is strictly monotonic increasing } h = \ln$$

Discriminant Function:

$$g(x) := \ln\{P(x|c_j)P(c_j)\}, j \in \mathcal{C}$$

Goal: given x , find $\arg \max_{j \in \mathcal{C}} g_j(x)$ By applying the normal distribution we obtain

$$\ln P(c_j) - \ln\{(2\pi)^{d/2}|\Sigma_j|^{1/2}\} - \frac{1}{2}(x - \mu)^\top \sum_j^{-1}(x - \mu) \sim \ln P(c_j) - \frac{1}{2}\{\ln |\Sigma_j| + (x - \mu_j)^\top \Sigma_j^{-1}(x - \mu_j)\}$$

which is a quadratic form in d dimensions and not depending on j .

To simplify we can assume:

- If $\Sigma_1 = \dots = \Sigma_c = \Sigma$ we obtain
 - Statistically: all covariances values are all the same. The only difference is in the means.
 - Geometrically: all the form of the covariances are equal.

$$\implies g_j(x) = w_j x + x_{j0} \quad w := \Sigma^{-1} \mu_j \quad w_{j0} = -\frac{1}{2} \mu_j^\top \Sigma^{-1} \mu_j + \ln P(c_j)$$

- If $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$, only when they are Gaussian we can assume that:
 - Statistically: all variables are statistically independent (taken two at a time).
 - Geometrically: different classes are ellipses with the axes parallel to the coordinate system.

$$\implies g_j(x) = \dots \text{to complete}$$

- If σ_j^2 are all equal, so $\Sigma = \sigma^2 I$. This means:

- Statistically: $\dots \text{to complete}$
- Geometrically: $\dots \text{to complete}$

$$\implies g_j(x) = \dots \text{to complete}$$

- If $P(c_j)$ are all equal $= \frac{1}{|\mathcal{C}|}$. This means:

- Statistically: $\dots \text{to complete}$
- Geometrically: $\dots \text{to complete}$

$$\implies g_j(x) = -\|x - \mu_j\|^2$$

4.1 K-Nearest Neighbours (KNN)

We start with a training set D^n , fix a positive integer K (hyper-parameter). The predicted class of x is the majority class among the k data points in D^n closest to x . Ties are broken randomly, are not possible in binary classification when K is odd but in multiclass-classification are unavoidable.

Cover & Hart Theorem: Given a binary classification and a $1-NN$. Let p_{bay} is the probability of error of the Bayes classifier and P_{1NN} the one for the $1-NN$. Consider a training set of size n

$$\text{as } n \rightarrow \infty, P_{bay} \leq P_{1NN} \leq 2P_{bay}(1 - P_{bay}) \quad \text{assumes } d(x, z) = \|x - z\|_2$$

Which metric should we use? We generalize the metric by giving weight to the partial computations:

$$\sqrt{(x - z)^\top A (x - z)}, \text{ when } A \text{ is positive definite } A = I \rightarrow \|x - z\|_2$$

We can learn A from data (metric learning). How we ensure the positive-definiteness of A ?

- Every matrix constructed as $A = B^\top B$ has this properties. We need to learn B .
- $(x - z)^\top A (x - z) = (x - z)^\top B^\top B (x - z) = [B(x - z)]^\top [B(x - z)]$ this is equal to preprocess all data points as $r(x) = Bx \rightarrow (r(x) - r(z))^\top (r(x) - r(z))$.

We can identify $A = \Sigma \rightarrow$ choosing Mahalanobis metric.

Clarke's Distance:

$$d(x, z) = \frac{|x - z|}{x + z}$$

This is a semantic distance, the euclidean distance is an additive distance and is not adapt for every situation.

It is known that, for an $x \in \mathbb{R}^d$, a local estimate of the density of x :

$$\hat{p}(x) = \frac{k}{n} V_d^{-1}$$

where V_d is the volume of the smallest hypersphere centered at x and containing exactly the k -nearest neighbours of x in D^n .

- k_j = number of nearest neighbours with class label c_j .
- n_j = number of data points with class label c_j in the data.

$$\sum_{j=1}^c n_j = n$$

$$\bullet \quad \hat{p}(x|c_j) = \frac{k_j}{n_j} \cdot V_d^{-1} \quad \hat{p}(x) = \frac{k}{n} \cdot V_d^{-1} \quad \hat{p}(c_j) = \frac{n_j}{n}$$

$$\hat{p}(c_j|x) = \frac{k_j}{k}$$

5 Perceptron (Rosenblatt 58)

$$f(x) = \text{sign}(w \cdot x + w_0) \quad w, x \in \mathbb{R}^d, w_0 \in \mathbb{R}$$

- $w \perp \pi$

$$\bullet \quad w := \begin{pmatrix} w_0 \\ \dots \\ w_n \end{pmatrix}; y := \begin{pmatrix} y_1 \\ \dots \\ y_3 \end{pmatrix}; x^i := \begin{pmatrix} 1 \\ x_1 \\ \dots \\ x_n \end{pmatrix} \rightarrow f(x) = \text{sign}(w \cdot x)$$

Linsep: if $\exists w \in \mathbb{R}^{d+}$ such that $\forall i : 1 \leq i \leq n : y^i(w \cdot x^i) > 0$.

μ -Linsep: if $\exists w \in \mathbb{R}^{d+}$ such that $\forall i : 1 \leq i \leq n : \mu > 0$.

Algorithm 1 Perceptron (D^n, α)

```

1: Initialize  $\mathbf{w}^{(0)}, t \leftarrow 0$ 
2: repeat
3:   mistakes  $\leftarrow$  False
4:   for  $i = 1$  to  $n$  do
5:      $\hat{y}^{(i)} \leftarrow \text{sign}(\mathbf{w}^{(t)} \cdot \mathbf{x}^{(i)})$   $\triangleright$  current prediction
6:     if  $\hat{y}^{(i)} \neq y^{(i)}$  then
7:       mistakes  $\leftarrow$  True
8:        $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + \alpha y^{(i)} \mathbf{x}^{(i)}$ 
9:        $t \leftarrow t + 1$ 
10:    end if
11:  end for
12: until no mistakes
13: return  $\mathbf{w}^{(t)}$ 

```

Observations:

- At each iteration, an example (x^i, y^i) is misclassified if $y^i(w(t) \cdot x^i) \leq 0$

$$y^i(w(t+1) \cdot x^i) = y^i([w(t) + \alpha y^i x^i] \cdot x^i) = y^i(w(t) \cdot x^i) + \alpha \|x^i\|^2 > y^i(w(t) \cdot x^i)$$

- We can take $\alpha = 1$ and take it off.

Theorem (Novikoff 62): Given D^n μ -linsep, this means there exists some $w^* \in \mathbb{R}^{d+1}$ s.t. $\forall i : 1 \leq i \leq n : \mu = \frac{y^i(w^* \cdot x^i)}{\|w^*\|}$. Then the number of correction steps needed by the Perceptron algorithm to find a linsep solution is $t \leq \frac{r^2}{\mu^2}$ where $r \geq \max(\|x^i\|^2, i = 1, \dots, n)$

Observations:

- Pre-processing x^i s.t. $\frac{x^i}{\|x^i\|} \rightarrow r = 1$. There is no better bound than this and the bound is tight.
- Remarkably, the bound does not depend on d .
- Remarkably, the bound does not depend on n .
- Remarkably, the bound does not depend on order of presentation of the data in D^n .
- Larger μ may lead to faster convergence.

5.1 Dual Perceptron

Proposition: If $w(0) = 0$, then the final solution $w^* = \sum_{i=1}^n \alpha_i^* y^i x^i$, where α_i^* = number of times (x^i, y^i) has been misclassified and w^* is $w(t)$ on the final t (last w bound).

$$f(x) = \text{sign}(w^* \cdot x) = \text{sign}\left(\left(\sum_{i=1}^n d_i^* y^i x^i\right) \cdot x\right) = \text{sign}\left(\sum_{i=1}^n d_i^* y^i (x^i \cdot x)\right) \quad \text{DUAL FORMULATION}$$

Algorithm 2 Dual Perceptron (D^n , linsup)

```
1:  $\alpha := 0$ ;  $t := 0$ 
2: repeat
3:   mistakes := False
4:   for  $i := 1$  to  $n$  do
5:      $\hat{y}^i := \text{sign}\left(\sum_{j=1}^n \alpha_j y^j (x^j \cdot x^i)\right)$ 
6:     if  $\hat{y}^i \neq y^i$  then
7:       mistakes := True
8:        $\alpha_i := \alpha_i + 1$ 
9:        $t := t + 1$ 
10:    end if
11:  end for
12: until not mistakes
13: return  $\alpha$ 
```

Observations:

- Both primal and dual are the same model.
- $G_{ij} = x^i \cdot x^j \leftarrow$ Gram matrix.
- It is expected that α^* is sparse (a good fraction of $\alpha_i^* = 0$).

5.2 The Delta Rule

The least square problem:

$$D^n := \{(x^1, y^1), \dots, (x^n, y^n)\}, x^i \in \mathbb{R}^{d+1}, y^i \in \mathbb{R}$$

Gather

$$X_{n \times (d+1)} \text{ matrix} = \begin{pmatrix} 1 & \dots \\ 1 & \dots \\ 1 & \dots \\ 1 & \dots \end{pmatrix}; y = \begin{pmatrix} y^1 \\ y^2 \\ \dots \\ y^n \end{pmatrix}$$

With $n \gg d$: we have an overdetermined system. The LSP amounts to $\frac{\min_{w \in \mathbb{R}^{d+1}}}{2} (y - Xw) \cdot (y - Xw) = \frac{1}{2} \|y - Xw\|^2$.

Let $A = X^\top X$ is $(d+1) \times (d+1)$ we have two cases: a) A is full rank ($\text{rank} = d+1$); b) A is rank deficient.

In either case, we have to solve

$$X^\top X w = X^\top y \quad (\text{Gauss's Normal Equations})$$

- (a) $\hat{w} = (X^\top X)^{-1} X^\top y$, pseudo-inverse of X (Moore-Penrose). When X is square and non-singular, the pseudo-inverse coincides with the inverse ($X^\# = X^{-1}$). The solution is unique.
- (b) $X^\top X$ is not full-rank, we have an infinite number of solutions, but there is one $\hat{w}$ having the smallest $\|\hat{w}\|$.

$$E(w) = \frac{1}{2} \|y - Xw\|^2 = \frac{1}{2} \sum_{j=1}^{d+1} (y^j - (\sum_{i=1}^{d+1} w_i \cdot x_j^i))^2$$

- For the Gradient Descent we need

$$j = 1, \dots, d+1 \quad \frac{\partial E}{\partial w_j} = - \sum_{i=1}^n (y^i - w \cdot x^i) x_j^i$$

$$\Delta w_j = -\eta \sum_{i=1}^n (y^i - w \cdot x^i) x_j^i, \eta > 0 \quad \implies \quad (y^i - w \cdot x^i) = \delta$$

If $\eta > 0$ is chosen sufficiently small, then $w(t) \xrightarrow{n \rightarrow \infty} w^*$

TH: If a is chosen sufficiently small η is $\eta < \frac{\alpha}{\lambda_{max}}$, λ_{max} is largest eigenvalue of the input auto-correlation matrix: $R = \mathbb{E}(xx^\top) \approx \frac{1}{n} \sum_{i=1}^n x^i (x^i)^\top$.

Epoch: present to the network a number $n_0 \leq n$ of the data points. If $n_0 = n \rightarrow$ batch, $n_0 = 1 \rightarrow$ on-line, $1 < n_0 < n \rightarrow$ minibatch (the last two are an example of SGD).

- On-Line learning: start with $w(0)$ and choose a number randomly $\eta(t) \in \{1, 2, \dots, n\}$

$$\Delta w_j = \eta(t)(y^{\eta(t)} - w \cdot x^{\eta(t)})x_j^{\eta(t)}$$

TH: If the sequence $\eta(t)$ satisfies:

1. $\sum_{t \geq 0} \eta(t) = \infty$
2. $\sum_{t \geq 0} \eta^2(t) < \infty$

Then $w(t)$ converge to w^* in the mean sequence sense:

$$\lim_{t \rightarrow \infty} \|w(t) - w^*\|^2 = 0$$

6 Layer Types: UA Property

Feed-Forward Neural Networks (FFNN): has no cycles, is a labelled and directed graph. The layer types are:

- Perceptron (hesinside)
- Perceptron (sigmoids)
- Perceptron (ReLU)
- Convolutional (+ pooling)
- Attention
- Output
- Radial Basis Function

Let $k \subset \mathbb{R}^d$ be a compact set. We denote by $C(K)$ the set of continuous function in $K, f : K \rightarrow \mathbb{R}$, equipped with the sup-norm: $\sup_{x \in K} |f(x)| = \|f\|_\infty$.

We say that a class of functions $\mathcal{F}$ is dense in $C(K)$ when $\forall f \in C(K), \forall \varepsilon > 0, \exists g \in \mathcal{F}$ s.t. $\|f - g\|_\infty < \varepsilon$.

1 Hidden Layer MLP:

$$g(x) = \sum_{i=1}^h w_i \cdot \sigma(v^i \cdot x + b_i)$$

- $\sigma(t) = \text{sigmoid (logistic, tanh, ...)}$.
- v^i is the weight vector of hidden neuron i .
- b_i is the bias term of hidden neuron i .

TH 1 (Cybenko 89): FFNN with 1 hidden layer, with any sigmoid are dense in $C(K)$

$$\forall f \in C(K), \exists w \in \mathbb{R}^h, \{v^1, \dots, v^h\} \in \mathbb{R}^d, b \in \mathbb{R}^h \text{ s.t. } \|g - f\| < \varepsilon, \forall \varepsilon > 0$$

TH 2 (Hornik et al 89): Same as TH1, σ can be any continuous, bounded, non constant.

TH 3 (Lashno et al 93): Same as TH1, $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ that is continuous and is not a polynomial.

7 VC-Dimension Theory (Binary Classifications)

Is the standard way of measuring the complexity of a function class $\mathcal{F}$. Our goal is to formalize:

$$R(f) \leq \hat{R}_{emp}(f, D_n) + \text{complexity_term}(f, n, \nu) \rightarrow \text{TrueError} \leq \text{TrainingError} + \text{Complexity}$$

Shatter: we say that a function class $\mathcal{F}$ shatters a set $\{x^1, \dots, x^n\}, x^i \in \mathbb{R}^d$ if, for every possible labelling (there are 2^n possible assignment of labels ± 1 to the x^i), we can always find a functions $f \in \mathcal{F}$ that realizes (makes no errors) the assignment.

VC-Dim: the VC dimension of a class function $\mathcal{F}$ (denoted by ν) is the size of the largest set that can be shattered by $\mathcal{F}$.

To conclude that $VC - \dim(\mathcal{F}) = \nu$, it takes two:

- $\exists \{x^1, \dots, x^n\}$ that can be shattered by $\mathcal{F} \implies VC - \dim(\mathcal{F}) \geq \nu$
- $\nexists \{x^1, \dots, x^n\}$ that can be shattered by $\mathcal{F} \implies VC - \dim(\mathcal{F}) < \nu + 1$

When the model is very simple, the VC-dim is going to be really close to the number of parameters. A model can be:

- **Parametric**: first we fix a certain structure and second we compute/optimize the parameters.
- **Non-Parametric**: such structure is not fixed a priori but also optimized.

Theorem: Let $\mathcal{F}$ a class of function taking values in $\{+1, -1\}$ with VC-dim ν . Then, $\forall \delta > 0$, with probability at least $1 - \delta$:

$$\forall f \in \mathcal{F}, R[f] \leq \hat{R}_{emp}(f, D_n) + A(n, \nu, \delta)$$

where

$$A(n, \nu, \delta) = \sqrt{\frac{2\nu \ln(\frac{ne}{\nu})}{n}} + \sqrt{\frac{\ln(\frac{1}{\delta})}{2n}}$$

The bound shows that the critical quantity is the ratio $\frac{n}{\nu}(\uparrow)$.

$$VC - \dim(\mathcal{F}_{lim}^d) = d + 1$$

7.1 Application to SVM

- In the primal formulation $\mathcal{F}_{lim}^d := \{x \rightarrow \text{sign}(w \cdot x + b), w, x \in \mathbb{R}^d, b \in \mathbb{R}\}$
- The margin $\mu := \arg \max_{w \in \mathbb{R}^d} \frac{1}{\|w\|}$ subject to $y^i(w \cdot x^i + b) \geq 1$
- D_n is linearly separable $\implies f_{svm}(\text{hard})$
- The loss of the SVM is the Hinge Loss: $\max\{0, 1 - y^i(w \cdot x^i + b)\}$, smallest convex upper-bound on $\ell_{0/1}$.

We need to convert this problem into the dual form:

1. Obtain the Lagrangian:

$$\mathcal{L}(w, b) = \frac{1}{\|w\|} - \sum_{i=1}^n \alpha_i [y^i(w \cdot x^i + b) - 1]$$

2. Compute the gradient of the Lagrangian wrt $w, b = 0$:

$$w = \sum_{i=1}^n \alpha_i y^i x^i \quad \text{and} \quad \sum_{i=1}^n y^i x^i = 0$$

3. $\max_{\alpha \in \mathbb{R}^n} \mathcal{L}_w(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y^i y^j (x^i \cdot x^j)$ subject to $\alpha_i \geq 0, \sum_{i=1}^n y^i x^i = 0$

$\mathcal{L}_w$ is infinity-differentiable, quadratic, convex, positive, with hessian $H = (y^i x^i \cdot y^j x^j)_{i,j}$.

$$f_{svm}(x) = \text{sign}(w^* \cdot x + b^*) \rightarrow w^* = \sum_{i=1}^n \alpha_i y^i x^i, \alpha_i \geq 0 \rightarrow \text{sign}\left(\sum_{i=1}^n \alpha_i y^i (x^i \cdot x) + b\right)$$

Observation: Can we obtain the margin μ ? $\mu = \sqrt{\frac{1}{\|\alpha\|_1}}$, but is valid only for hard SVM.

7.1.1 Quasi Linsep Case

Quasi Linsep:

$$R(f) \leq TR_{err}(f, D_n) + A(n, \delta, \nu), f \in \mathcal{F} \quad \text{replace} \quad VCdim(\mathcal{F}_{lin}^d) = \nu \rightarrow d + 1$$

TH: Let D_n a dataset. Define a subclass

$$\mathcal{F}_\mu^d := \{x \rightarrow \text{sgn}(w \cdot x + b), w, x \in \mathbb{R}^d, b \in \mathbb{R}, \mu > 0, \|w\| \leq \frac{1}{\mu}\} \subset \mathcal{F}_{lin}^d$$

Then

$$VCdim(\mathcal{F}_\mu^d) \leq \min\{\lceil \frac{R^2}{\mu} \rceil, d\} + 1$$

Where R is the radius of the smallest hyperplane centred at the origin and containing all the x^i data points.

For the Primal SVM we obtain

$$\max_{w \in \mathbb{R}^d, b \in \mathbb{R}, \xi \in \mathbb{R}^n} \left\{ \frac{1}{\|w\|} - C \sum_{i=1}^n |\xi_i| \right\}$$

For the Dual SVM we obtain

$$\min_{\alpha \in \mathbb{R}^n} \mathcal{L}_w(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y^i y^j (x^i \cdot x^j) \quad \text{s.t.} \quad C \geq \alpha_i \geq 0, i = 1, \dots, n$$

$$C \rightarrow 0 \rightarrow \text{underfitting} \quad C \rightarrow \infty \rightarrow \text{overfitting}$$

8 Learning in Functional Space

Hilbert Space:

- vector space ($\mathbb{R}^d$), inner product (dot product)
- norm, metric (distance)
- vector spaces of dimension $d \in \mathbb{N}^+ \cup \{\infty\}$ are called pre-Hilbert spaces.
 - Ex of inner product: $\mathbb{R}^d \quad x \cdot y = \sum_i x^i y^i$
 - Ex $C([a, b])$ set of real continuous functions in $[a, b] \rightarrow \langle a, b \rangle = \int_a^b f(x)g(x) dx$
 - * $L^2(a, b)$ space of real functions that are measurable and square-integrable.
 - * A succession $\{x_n\}$ is convergence to a point x (limit) if $d(x_n, x) = \|x_n - x\| \xrightarrow{n \uparrow \infty} 0$.

Def: $\{x_n\}$ is convergent to x if

$$\forall \epsilon > 0, \exists n, \|x_n - x\| \leq \epsilon$$

- * A succession $\{x_n\}$ is Cauchy convergent if $\|x_n - x_m\| \xrightarrow{n, m \geq N, as N \rightarrow \infty} 0$.

Def: $\{x_n\}$ is Cauchy convergent if

$$\forall \epsilon > 0, \exists n, \forall m, \|x_n - x_m\| < \epsilon$$

- * In $\mathbb{R}^d$: *convergent* $\Leftrightarrow$ *Cauchy convergent*.
- * In a general metric space *Cauchy convergence* $\leftarrow$ *convergent*.

Completeness: a metric space is complete if all Cauchy sequences are convergent.

Hilbert Space: a space of functions endowed with an inner product that is complete under the norm induced by the inner product

$$ip \rightarrow \| \rightarrow distance \rightarrow Cauchy sequence \rightarrow convergence$$

Kernel Function: any function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ satisfying:

- $k(x, x') = k(x', x)$
- Pseudosemidefinit-ness:

$$\forall n \in \mathbb{N}^+, \forall \{x^1, \dots, x^n\}, x^i \in \mathcal{X}, z^\top K z = \sum_{i=1}^n \sum_{j=1}^n z_i K_{ij} z_j \geq 0 \quad K_{ij} := k(x^i, x^j)$$

RKHS (Reproducing Kernel Hilbert Spaces:) a Hilbert Space $\mathcal{H}$ of functions definite on $\mathcal{X} : f \in \mathcal{H}$ are of the form $f : \mathcal{X} \rightarrow \mathbb{R}$ is called a RKHS if $\exists k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ s.t.:

- The functions $k(\cdot, x) \in \mathcal{H}, \forall x \in \mathcal{X}$.
- $\forall x \in \mathcal{X}, \langle k(\cdot, x), f \rangle_H = f(x), \forall f \in \mathcal{H}$

Moore-Aronszajn Theorem: every kernel function k generates a unique RKHS $\mathcal{H}_k$ having k as its reproducing kernel, and viceversa.

We can express this problem as an optimization problem:

$$\min_{f \in \mathcal{H}_k} \left\{ \frac{1}{n} \sum_{i=1}^n \ell(y^i, f(x^i)) + \lambda \|f\|_{H_k}^2 \right\} \quad \|f\|_{H_k}^2 = \langle f, f \rangle_H$$

Corollary: Given a kernel function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, there exists $\mathcal{H}_k$, the RKHS generated by k , s.t. $\exists \phi : \mathcal{X} \rightarrow \mathcal{H}_k$:

$$\langle \phi(x), \phi(x') \rangle_{\mathcal{H}_k} = k(x, x'), \forall x, x' \in \mathcal{X}$$

This is known as the *kernel trick*, meaning that, if we know k , we don't need to compute the ϕ 's. A Kernel function can be seen as:

- Inner product in some space.
- Positive semidefinite and symmetric function.
- Similarity function (opposite of a metric distance).

Equivalent Definition of psd-ness:

- All the eigenvalues are ≥ 0
- $\forall z \in \mathbb{R}^n, z^\top K z \geq 0$

Proposition:

$$k(x, x') = (x^\top x' + c), \quad x, x' \in \mathbb{R}^q, c \geq 0 \in \mathbb{R}$$

is a kernel function and is called polynomial kernel.

The ϕ generates one feature for every possible monomial up to degree $q(0, 1, 2, \dots, q)$ with d variables $(x_1, \dots, x_d)$.

The Representation Theorem: Let $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be a psd and symmetric kernel and $\mathcal{H}_k$ its RKHS. Let $D_n = \{(x^1, y^1), \dots, (x^n, y^n)\}, x^i \in \mathcal{X}, y^i \in \mathcal{Y}$ a TR set. Let $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_o^+$ be a suitable loss function, used as $\ell(y, \hat{y})$, where $\hat{y}$ is the prediction and y is the value in the TR set, and call w a set of learnable parameters. Consider an optimization problem:

$$\arg \min_{f \in \mathcal{H}_k} \left\{ \frac{1}{n} \sum_{i=1}^n \ell(y^i, f_w(x^i)) + \lambda \Omega(\|f_w\|_{\mathcal{H}_k}) \right\}, \lambda \geq 0$$

Ω is a non-decreasing function $\omega : \mathbb{R}^+ \rightarrow \mathbb{R}^+$.

Then the problem admits a solution of the form:

$$\hat{f}_\lambda = \sum_{i=1}^n \alpha_i \cdot k_{x^i} \|\hat{f}_\lambda(\cdot)\| = \sum_{i=1}^n \alpha_i k(x^i, \cdot) \|\hat{f}_\lambda(x)\| = \sum_{i=1}^n \alpha_i k(x^i, x)$$

- If, further, Ω is assumed to be increasing, then any solution is of the form.
- If, further, ℓ is taken to be convex wrt w , then the solution is unique.

8.1 Kernel Properties

Given $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, the following properties hold the kernel:

1. Scalar product: $a \cdot k, a \geq 0 \in \mathbb{R}$
2. Sum: $k_1 + k_2$
3. Product: $k_1 \cdot k_2$
4. Combinations: $\sum_{i=1}^n w_i k_i, w_i \geq 0$
5. $k_{normalization}(x, y) = \frac{k(x, y)}{\sqrt{k(x, x)} \cdot \sqrt{k(y, y)}} \in [-1, 1]$, this corresponds to the angle between x, y in H_k .
6. Pointwise convergence: $k_1, \dots, k_n$, then $\lim_{n \rightarrow \infty} k_n(x, y) = k(x, y), \forall x, y \in \mathcal{X}$.
7. Composition: given k and $f : \mathbb{R} \rightarrow \mathbb{R}$, then $(f \circ k)(x, y) = f(k(x, y))$
Schöenberg '42: Let $f(t) = \sum_{n=0}^{\infty} a_n t^n$ be analytic on $(-r, r), r > 0$. If $a_n \geq 0$, for any $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ $f \circ k$ is a valid kernel:
 - $f(t) = e^t, a_n = \frac{1}{n!} > 0$
 - $f(t) = \frac{1}{1-t} = \sum_{n=0}^{\infty} t^n, |t| < 1$
8. Given $g : \mathbb{R} \rightarrow \mathbb{R}$ any function, $g(x) \cdot k(x, y) \cdot g(y)$
9. Direct sum: $k_1 : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}, k_2 : \mathcal{Z} \times \mathcal{Z} \rightarrow \mathbb{R}$ we obtain $k((x_1, z_1), (x_2, z_2)) = k_1(x_1, x_2) + k_2(z_1, z_2)$
10. Tensor product: $k((x_1, z_1), (x_2, z_2)) = k_1(x_1, x_2) \cdot k_2(z_1, z_2)$

8.2 Kernel Functions for Non-Vectorial Data

We have data where $\mathcal{X} \neq \mathbb{R}^d$, we have some cases:

1. $\mathcal{X} = SETS$, subsets of a finite universal set. You can use the *Jaccard similarity*, that given $A, B \subseteq U$, $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$. This has some properties:

- $J(A, A) = 1 \iff A = B$
- $J(A, B) = J(B, A)$
- $J \in [0, 1]$
- $J(A, B) = 0 \iff A \cap B = \emptyset$

This measure ignores the 0-0 matches and cares about the 1-1 matches.

2. $\mathcal{G} = GRAPHS$, we are gonna define kernels from graphs $\rightarrow \mathbb{R}$. We are gonna compute a similarity between two nodes using the structure of relationships of the graph.

- (a) Come up with a similarity (symmetry) s between nodes. Out of it we can create a similarity matrix S :

$$S_{ij} = s(x^i, x^j), x^i, x^j \in \mathcal{X}(o)$$

If you don't know s , the most basic $S = W$, the connectivity matrix of the graph:

$$S_{ij} = W_{ij} = \mathbb{1}_{o^i \sim o^j}$$

S^ℓ is the matrix of paths of length ℓ between nodes.

Lemma: if $S_{n \times n}$ is symmetric, then $e^{\gamma S}$ is positive definite.

- (b)

$$K = e^{\gamma S}, \forall \gamma > 0$$

8.3 Polynomial Kernel in SVM vs Linear Regression

What is the difference between a polynomial kernel in SVM and a linear regression creating by hand all the monomials? They produce the same feature space, but differ in how is computed, optimized and regularized.

SVM-Poly ($q=2$)	Linear Regression (ridge)
Learning parameters are in the training set	Learns parameters from the variables $V_1, V_2, V_3 = V_1 - V_2, V_4 = V_1^2, \dots$

8.4 Design of (Metric) Distances and Similarities

Call $\mathcal{X}$ the data space, and a similarity measure $s : \mathcal{X} \times \mathcal{X} \rightarrow [0, 1]$.

Similarity Measure:

- $s(x, x) = 1 \iff x = y$
- $s(x, y) = s(y, x)$
- $\mathcal{T}[s(x, z), s(z, y)] \geq s(x, y)$, where $\mathcal{T}$ are symmetry, transitivity or reflexivity.

Gowen Similarity:

$$s_G(x^i, x^j) = \frac{\sum_{k=1}^d \delta_{ijk} s_{ijk}}{\sum_{k=1}^d w_k \delta_{ijk}}$$

where we have

- w_k is learnt from the data.
- $s_{ijk} := s_k(x_k^i, x_k^j)$.
- s_k are partial similarities that compare delta objects using variable k .

- $\delta_{ijk} : \begin{cases} 1, & \text{if } s_{ijk} \text{ can be computed} \\ 0 & \text{otherwise} \end{cases}$

We can have other cases, when k is:

- *Bitstring or Set*: $s(x_k^i, x_k^j)$ becomes the Jaccard similarity.
- *Categorical*: $s(x_k^i, x_k^j) = \begin{cases} 1, & x_k^i = x_k^j \\ 0, & \text{otherwise} \end{cases}$

Now we can build a matrix S_G where each cell is (i, j) is $s_G(x^i, x^j)$

TH: If there are no missing values, the matrix S_G is psd and symmetric.

Proposition: Gowen method is equivalent to assign the missing similarity the average of the non-missing.

Euclidean: suppose D is a distance matrix for a certain dataset, D is said to be Euclidean if n points in $\mathbb{R}^d$, if some ℓ can be found s.t. $\|z^i - z^j\| = D_{ij}$.

Suppose $\mathbb{1}$ is the matrix of all ones and $\mathbf{1}$ is the vector of all ones. Δ is the matrix with elements $-\frac{1}{2}D_{ij}^2$.

TH: D is Euclidean $\iff$ the matrix $(\mathbb{1} - \mathbf{1}s^T)\Delta(\mathbb{1} - s\mathbf{1}^T)$ is psd, where $s^\top \mathbf{1} = 1$.

TH: If S is a psd simmetryc matrix, with elements $0 \leq s_{ij} \leq 1, s_{ij} = 1$, then the dissimilarity matrix with elements $d_{ij} = \sqrt{1 - s_{ij}}$ is Euclidean.

9 Hopfield Networks

9.1 Introduction and Core Concepts

A Hopfield network is a single-layer recurrent neural network where the only layer consists of n neurons (equal to the input dimension). It is primarily designed to function as a **Content-Addressable Autoassociative Memory**.

- **Autoassociativity:** When the network is presented with a new pattern (input) Ψ , the output converges to the stored key in the memory set L that is closest to Ψ (in terms of Hamming distance).
- **Robustness:** The network deals with incomplete or erroneous keys. It can reconstruct a complete pattern from a fragment or correct a noisy input.
- **Analogy:** Similar to a mechanical system that tends toward minimum-energy states (attractors), a Hopfield network evolves toward stored patterns which act as low-energy attractors.

9.2 Architecture and State

- **State Representation:** The state of the network is the vector of activations of the n neurons. The state space can be defined as $\{0, 1\}^n$ or, more conveniently for analysis, $\{-1, +1\}^n$ via the transformation $s = 2e - 1$.
- **Dynamics:** The network "traverses" the hypercube of states until it stabilizes.

9.3 Formal Analysis and Dynamics

9.3.1 Neuron Update Rule

The state s_i of neuron i is updated based on the inputs from all other neurons j . The update rule is:

$$s_i := \text{sgn} \left(\sum_{j=1}^n w_{ij} s_j - \theta_i \right)$$

Where:

- w_{ij} is the weight of the connection between neuron j and neuron i .
- θ_i is the threshold (often set to 0 for simplicity).
- The sign function is defined as: $\text{sgn}(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

9.3.2 Update Modes

1. **Synchronous:** All neurons update their states simultaneously.
2. **Asynchronous:** Neurons update sequentially in a random order or according to a fair distribution. (This is the standard assumption for stability analysis).

The process stops when the network reaches a **stable state** (no more changes occur).

9.4 Hebbian Learning (Storing Memories)

To store a set of patterns $L = \{\xi_k\}$, we define the weight matrix W .

9.4.1 Single Pattern Case ($l = 1$)

For a single pattern ξ , we set weights such that the pattern is stable ($s_i = \xi_i$).

$$w_{ij} = \frac{1}{n} \xi_i \xi_j$$

This is based on the outer product rule.

9.4.2 Multiple Patterns Case ($l \geq 1$)

For l patterns, the weights are the superposition of individual pattern weights:

$$w_{ij} = \frac{1}{n} \sum_{k=1}^l \xi_{ki} \xi_{kj}$$

In matrix notation, if $E = [\xi_1, \dots, \xi_l]$, then:

$$W_{n \times n} = \frac{1}{n} E E^t$$

Note: W is a symmetric matrix.

9.5 Stability Analysis

For a stored pattern ξ_v to be a stable attractor, the local field h_{vi} must have the same sign as the pattern bit ξ_{vi} .

$$h_{vi} = \sum_{j=1}^n w_{ij} \xi_{vj} = \xi_{vi} + \text{crosstalk}(v, i)$$

where the crosstalk term is:

$$\text{crosstalk}(v, i) = \frac{1}{n} \sum_{j=1}^n \sum_{k \neq v} \xi_{ki} \xi_{kj} \xi_{vj}$$

If $|\text{crosstalk}(v, i)| < 1$, the bit remains stable. As the number of stored patterns l increases, the crosstalk noise increases, eventually destabilizing the memory.

9.6 Capacity

What is the maximum number of patterns (l_{max}) a network of n neurons can store?

- **Perfect Recall:** $l_{max} \approx \frac{n}{2 \ln(n)}$.
- **With Small Errors ($\approx 1.6\%$ error rate):** $l_{max} \approx 0.138n$.

Ideally, patterns should be orthogonal or negatively correlated to maximize capacity.

9.7 Spurious States

The network often contains stable states that were not explicitly stored, called spurious states.

- **Reversed Patterns:** If ξ is stored, $-\xi$ is also stored (same energy).
- **Mixture States:** Linear combinations of an odd number of patterns can create stable mixture states (e.g., $\text{sgn}(\xi_a + \xi_b + \xi_c)$).
- **Mitigation:** Setting $w_{ii} = 0$ (zero diagonal) helps make the network more stable and reduces spurious states.

9.8 The Energy Function

For symmetric weights ($w_{ij} = w_{ji}$), there exists an energy function $H(s)$ that always decreases or remains constant as the system evolves (Lyapunov function).

$$H(s) = -\frac{1}{2} \sum_{i=1}^n \sum_{j \neq i}^n w_{ij} s_i s_j$$

- **Attractors:** The stored memories correspond to local minima of the energy function.
- **Convergence:** Since H is lower-bounded and decreases with every neuron update, the network is guaranteed to converge to a stable state.

9.9 Applications

9.9.1 Optimization Problems

Hopfield networks can solve combinatorial optimization problems (often NP-complete) by mapping the cost function of the problem to the energy function of the network.

- **Graph Bipartitioning:** Partitioning nodes into two sets with minimum crossing edges.
- **Traveling Salesman Problem (TSP):**
 - *Goal:* Shortest tour visiting n cities exactly once.
 - *Representation:* $n \times n$ neurons (city vs. position in tour).
 - *Energy Function Terms:*

$$H = \alpha(\text{Row Constraint}) + \beta(\text{Col Constraint}) + \gamma(\text{Global Sum}) + \epsilon(\text{Distance})$$

- *Issue:* High number of spurious states (invalid tours) often traps the network in local minima.

9.9.2 Pattern Reconstruction

- **Image Restoration:** Recovering clean images from noisy inputs (e.g., identifying numbers or concealed fighter aircraft shapes).
- **Example:** A network of 120 nodes (12×10 pixels) storing digit patterns.

10 Neural Networks Training → Kernel Functions

With 1 hidden layer perceptron, supposing that we are in "regression", we have

$$f(x) = b + \sum_{j=1}^H v_j \cdot \phi(\mu_j x + c_j)$$

where $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is the activation function.

Under a Bayesian framework, the learnable parameters are pdfs so we need priors on the parameters:

$$v_j \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \frac{\sigma_v^2}{H}) \quad u_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_u^2 I) \quad b \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_b^2)$$

but what happens when $H \rightarrow \infty$?

Neal '96: As $H \rightarrow \infty$, $f(x)$ converges in distribution to a GP with mean function $\mu(x) = 0$ and a covariance function $k(x, x') = \sigma_b^2 + \sigma_v^2 \mathbb{E}_{u,b}[\phi(\mu \cdot x + b) \cdot \phi(\mu x' + b)]$. Logistic $\rightarrow \text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$ is a proxy for the logistic $\rightarrow$ sigmoid.

Williams '96-'98: For 1 HL-MLPs with $\phi(z) = \text{erf}(z)$, $\mu \sim \mathcal{N}(0, \Sigma_u)$ over the augmented input $\tilde{x} = (bx_1 \dots x_d)$; the kernel function is

$$k(x, x') = \frac{2}{r} \arcsin\left[\frac{2\tilde{x}^\top \Sigma \tilde{x}'}{\sqrt{1 + 2\tilde{x}^\top \Sigma \tilde{x}} \sqrt{1 + 2\tilde{x}'^\top \Sigma \tilde{x}'}}\right] \rightarrow \text{stationary}$$

$$k_{RBF}(x, x') = \exp(-\gamma d_{man,z}(x, x')) \quad \gamma > 0 \quad \rightarrow \text{translational invariant}$$

Trick: "whitening", generalization of PCA to decorrelated variables but variances not equal to 1

$$\underset{\Sigma}{\text{input data } x} \xrightarrow{W} z, \mu = 1, \Sigma = \mathbb{1} \rightarrow d_{\Sigma}(x, x') = z^\top z$$

$$\text{Choi \& Saul '09: } k_n(x, x') = \int_{\mathbb{R}^+} \phi_{(w \cdot \tilde{x})} \phi(w \cdot \tilde{x}') \cdot (w \cdot \tilde{x})^n (w \cdot \tilde{x}')^n p(w) dw \quad \phi(z) = \frac{1}{2}[1 + \text{sgn}(z)] \text{ Heaviside}$$

- $n = 0$ Heaviside.
- $n = 1$ Relu $\rightarrow \max(0, z)$ $z = w \cdot \tilde{x}$
- $n = 2$ Relu² = quarterpipe.

Universal Approximation: let $\mathcal{H}$ be a RKHS defined by the kernel k_1 . For any compact $\mathcal{X} \in \mathbb{R}^d$ and any continuous function $\ell : \mathcal{X} \rightarrow \mathbb{R}$, there exists one h in $\mathcal{H}$ with $\sup_{x \in \mathcal{X}} |h(x) - \ell(x)| < \varepsilon; \forall \varepsilon > 0$.

Issues:

- This framework disregards the learning of parameters.
- What about multi-layer architectures? Depth should increase performance.

10.1 Kernel Functions → Neural Networks Training

The ℓ -fold composition of the RBF kernel becomes more trivial as $\ell \rightarrow \infty$. However, for $k = \arccos(n)$:

$$k_n^{(1)} = k_{\arccos}(x, x') \quad k_n^{\ell+1} = \frac{1}{\pi} [k_n^\ell(x, x') \cdot k_n^{(\ell)}(x, x')]^{\frac{n}{2}} \cdot J_n(\sigma_n^{(\ell)}) \quad \sigma_n^{(\ell)} = \arccos\left(\frac{k_n^{(\ell)}(x, x')}{\sqrt{k_n^{(\ell)}(x, x)} \cdot \sqrt{k_n^{(\ell)}(x', x')}}\right)$$

11 Graph Neural Networks

GNNs = NNs that operate on graph-structured data.

- Feature extraction from G .
- KNNs with graph kernels from/for graphs.
- GNNs for non-structured data (tabular data), sequences, trees, graphs.

GNNs can be used to make predictions at the edge/node/graph level, and to make classification.

Given $G = (V, E), |V| = n$, $A_{n \times n} = (A_{ij}) = \begin{cases} 1, & i \sim j \\ 0, & i \not\sim j \end{cases}$, $X_{n \times d} = \begin{pmatrix} \dots x_1^\top \dots \\ \dots \dots \dots \\ \dots x_n^\top \dots \end{pmatrix}; x^i \in \mathbb{R}^d$ Permutation Matrices:

one 1 in every row and one 1 in every column. So that the permutation $\tilde{A} = PAP^\top$.

Invariance Property: $GNN(\tilde{X}, \tilde{A}) = GNN(X, A)$, where $\tilde{X} = PX$ and P is the binary permutation.

An image is a particular case of a graph where $z_i^{(\ell+1)} = \phi(\sum_j w_j z_j^{(\ell)} + b)$, but this does not follow invariance. We have to do as follow:

$$z_i^{(\ell+1)} = \phi(w_{neigh} \sum_{j \in N(i)} z_j^{(\ell)} + w_{self} \cdot z_i^{(\ell)} + b)$$

in this case messages are activations.

GNN: sequence of layers $H^{(\ell)}$ that successively transforms the node embeddings. $H^{(\ell)}$ is the matrix of embeddings $= [h_1^{(\ell)}, \dots, h_n^{(\ell)}], \ell = 1, \dots, L$ and $h_i^{(\ell)} \in \mathbb{R}^d, i = 1, \dots, n$.

$$H^{(1)} := F(X, A, W^{(1)})$$

$$H^{(2)} := F(H^{(1)}, A, W^{(2)})$$

...

$$H^{(L)} := F(H^{(L-1)}, A, W^{(L-1)})$$

where $W^{(\ell)}$ are the parameters, for $\ell = 1, \dots, L$.

Algorithm 3 Message Passing Neural Network

Require: Graph $G = (V, E)$, node features $\{x_i\}_{i=1}^n$, number of layers L

Ensure: Node embeddings $\{h_i^{(L)}\}_{i=1}^n$

```

1: Initialize node representations:
2: for each node  $i \in V$  do
3:    $h_i^{(0)} \leftarrow x_i$ 
4: end for
5: for  $\ell = 0$  to  $L - 1$  do
6:   for each node  $i \in V$  do
7:     Aggregate:
8:      $z_i^{(\ell)} \leftarrow \frac{1}{|N(i)|} \sum_{j \in N(i)} \frac{h_j^{(\ell)}}{|N(j)|}$ 
9:     Update:
10:     $h_i^{(\ell+1)} \leftarrow \Phi\left(w_{self}^{(\ell)} h_i^{(\ell)} + w_{neigh}^{(\ell)} \sum_{j \in N(i)} h_j^{(\ell)} + b^{(\ell)}\right)$ 
11:   end for
12: end for
    return  $\{h_i^{(L)}\}_{i=1}^n$ 

```

The Φ is an element-wise ReLU.

12 Exam 2024

- **Group 0: SVM**

- (F) The SVM for classification is a regularized machine, and the penalized term is the margin of separation between the classes.
- (F) Increasing the value of C in a SVM, we reduce the chances of overfitting the data.
- (F) The architecture of a CNN is completely determined by the input image properties (image size and number of channels).
- (T) The kernel in a convolutional layer performs inner products.
- (F) The geometric margin of a support vector in a hard SVM is the maximum possible among all data vectors.

- **Group 1: GLMs**

- (T) These models are considered linear.
- (T) They are not regularized per se, but can be regularized by different norms.
- (T) They can deal with several (output) target types.
- (F) They always have a closed-form solution.

- **Group 2: Representer theorem**

- (T) It deals with possible minimizers of a regularized risk functional defined over a RKHS.
- (T) It expresses its solution as a finite linear combination of kernel functions (partially) evaluated on the known training data.
- (F) It allows the inclusion of learnable weights for the (implicitly) created features.
- (T) The RKHS is defined uniquely in terms of the given kernel function.

- **Group 3: Machine learning medley 1**

- (T) Bayesian methods can be used for dealing with hyperparameters in a principled way.
- (T) The risk of a model is the expected value of a chosen loss using this model as a predictor.
- (T) A Hopfield network iterates until it stabilises at a point which is a local minimum of its energy function.
- (F) The energy function of a Hopfield network must be seen as a function of the weights, given the state.

- **Group 4: MLPs and RBFs**

- (T) RBFs use one hidden layer, whereas MLPs use one or more.
- (F) RBFs are better extrapolators than interpolators, whereas for MLPs it is the other way around.
- (T) RBFs can use a principled variety of activation functions, whereas MLPs tend to use hand-crafted ones.
- (F) RBFs train way slower than MLPs, because one must take into account the cost of clustering.

- **Group 5: Machine learning medley 2**

- (F) A likelihood function must be seen as a function of the data, given the weights.
- (T) The chances of overfitting the training data can be lowered by increasing the training sample size.
- (F) The chances of overfitting the training data can be lowered by increasing the number of predictors.
- (T) The chances of overfitting the training data can be lowered by the inclusion of a penalty term on the norm of the model.

13 Exam 2025

- **1. General regression theory:** Consider an overfit regression model f . This implies that:
 - (T) Training error of f is 0.
 - (F) Bias (squared) of f is 0.
 - (F) Var of f is 0.
 - (F) MSE of f is 0.
 - (F) Risk of f is 0.
- **2. Decision tree class boundaries:** In standard binary decision trees (like CARTs), decision boundaries can be as general as:
 - (F) Hyperplanes
 - (T) Hyperboxes
 - (F) Hyperspheres
 - (T) Unions of hyperboxes
 - (F) Unions of hyperplanes
- **3. SVM class boundaries:** In Support Vector Machines created with the RBF kernel, decision boundaries (in the input space) can be as general as:
 - (F) Hyperplanes
 - (F) Hyperboxes
 - (T) Hyperspheres
 - (F) Unions of convex subspaces
 - (T) Arbitrarily complex subspaces
- **4. Kernels galore:** Let $k_1, \dots, k_n$ be kernels defined on the same set X . Which of the following expressions define valid kernels?
 - (T) $\sum_{i=1}^n \alpha_i k_i$, with $\sum_{i=1}^n \alpha_i = 1$
 - (T) $\prod_{i=1}^n (k_i)^{\alpha_i}$. $\alpha_i \in \mathbb{N}$ (taking $(k_i)^0 = 1$)
 - (F) $(k_1)^\gamma$, $\gamma \in \mathbb{R}^+$
 - (T) $\lim_{i \rightarrow \infty} k_i$, assuming the limit exists for all $x, x' \in \mathcal{X}$
 - (T) $\prod_{i=1}^n \exp\{\gamma_i k_i\}$ $\gamma_i \in \mathbb{R}^+$
- **5. On MLPs:** In MultiLayer Preceptrons
 - (F) The activation function of the hidden layers must be bounded
 - (F) The activation function of the hidden layers must be piece-wise linear
 - (T) All of the weights (including those in the output layer) are trained simultaneously
 - (T) The training error keeps going down as the number of epochs is increased (i.e, the training process keeps going on)
 - (F) The learning rate is an accelerator of the training process, with no influence on convergence
- **6. On Hopfield networks:** In a Hopfield Neural Network:
 - (T) As the network iterates, the energy function cannot increase
 - (T) The network tends to have a larger capacity (maximum number of desired memories that can be stored without recovery error) as the problem dimensionality increases
 - (F) Idem (same) as the more correlated the desired memories are
 - (T) The negative ("minus") of a desired memory will also be a desired memory
 - (T) In order to cast an optimization problem as an energy function, the former must be written as a quadratic form defined on the network states

- **7. VCdim rocks:** Let's consider the VC dimension of classes of margin-bounded SVM classifiers in $\mathbb{R}^d$, as seen in class
 - (T) The VCdim of each class is a finite natural number
 - (T) The VCdim of each class depends on d
 - (F) The VCdim of each class is equal to $d + 1$
 - (F) The VCdim of each class is upper-bounded by $d + 1$
 - (T) Larger bounds on the margin cannot increase the VCdim of the class
- **9. Kernel matrices:** What are necessary and sufficient conditions for the following 2×2 real matrix to be the kernel matrix of a normalized kernel?

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

- (F) $a = d = 1$
- (F) $b = c$
- (F) $a = d = 1$ and $b = c$
- (F) $|b| \leq 1$
- (T) $a = d = 1$ and $b = c$ and $|b| \leq 1$

14 Mock Test

- **Group 1: Bias-Variance Decomposition** Consider the bias-variance decomposition for a regression model f :
 - (T) The risk can be decomposed as $R[f] = (\text{bias}(f))^2 + \text{Var}(f) + \sigma^2$
 - (F) Increasing model complexity always decreases bias without affecting variance
 - (T) The irreducible error σ^2 represents the noise in the data
 - (T) $MSE(f) = [\text{Bias}(f)]^2 + \text{Var}(f)$
 - (F) A model with zero bias necessarily has zero variance
- **Group 2: Perceptron Algorithm** Regarding the Perceptron algorithm and its convergence:
 - (T) The Perceptron is guaranteed to converge if the data is linearly separable
 - (T) The convergence bound $t \leq r^2/\mu^2$ does not depend on the dimensionality d
 - (F) The order of presentation of training examples affects the convergence bound
 - (T) The dual formulation represents the solution as $w^* = \sum_{i=1}^n \alpha_i y^i x^i$
 - (F) The Perceptron algorithm requires the data to be μ -linearly separable with $\mu > 1$
- **Group 3: Bayesian Methods** In Bayesian approaches to machine learning:
 - (T) The posterior distribution $p(\theta|D^n)$ is proportional to $p(D^n|\theta)p(\theta)$
 - (F) The MAP estimator always produces the same result as the ML estimator
 - (T) The posterior predictive distribution integrates over parameter uncertainty
 - (T) In Bayesian linear regression with Gaussian priors, the posterior is also Gaussian
 - (F) The prior $p(\theta)$ has no influence when the dataset size $n \rightarrow \infty$
- **Group 4: GLMs (Generalized Linear Models)** About Generalized Linear Models:
 - (T) GLMs use a link function h to connect the linear predictor to the expected value
 - (T) GLMs are discriminative models that focus on $P(Y|X)$
 - (F) The noise distribution in GLMs must always be Gaussian
 - (T) GLMs can handle categorical variables natively
 - (T) The likelihood function in GLMs belongs to the exponential family
- **Group 5: K-Nearest Neighbors** Regarding the K-Nearest Neighbors algorithm:
 - (T) According to Cover & Hart theorem, $P_{1NN} \leq 2P_{\text{bayes}}(1 - P_{\text{bayes}})$ as $n \rightarrow \infty$
 - (F) KNN requires storing no training data once trained
 - (T) The estimate $\hat{p}(c_j|x) = k_j/k$ where k_j is the number of neighbors with class c_j
 - (T) Metric learning can improve KNN performance by learning the distance matrix A
 - (F) Using larger K always improves classification accuracy
- **Group 6: RKHS and Kernel Methods** For Reproducing Kernel Hilbert Spaces:
 - (T) Every kernel function k generates a unique RKHS $\mathcal{H}_k$
 - (T) The reproducing property states that $\langle k(\cdot, x), f \rangle = f(x)$ for all $f \in \mathcal{H}$
 - (F) A function is a valid kernel if and only if it is symmetric
 - (T) The kernel matrix K must be positive semidefinite
 - (T) The "kernel trick" allows computing inner products in high-dimensional spaces without explicit feature mapping
- **Group 7: Representer Theorem** The Representer Theorem states that:
 - (T) The solution has the form $\hat{f} = \sum_{i=1}^n \alpha_i k(x^i, \cdot)$

- (T) It applies to regularized risk minimization in RKHS
- (F) The solution always involves all n training points with non-zero coefficients
- (T) If the loss is convex, the solution is unique
- (T) The regularizer Ω must be non-decreasing
- **Group 8: Universal Approximation** Regarding universal approximation properties:
 - (T) 1-hidden-layer MLPs with sigmoid activations are dense in $C(K)$
 - (T) The activation function can be any continuous, bounded, non-constant function (Hornik et al.)
 - (F) Universal approximation guarantees that we can learn the optimal function efficiently
 - (T) The result holds for any continuous function that is not a polynomial (Leshno et al.)
 - (F) Universal approximation requires at least 2 hidden layers
- **Group 9: VC Dimension Theory** About VC dimension:
 - (T) $VCdim(\mathcal{F})$ is the size of the largest set that can be shattered by $\mathcal{F}$
 - (T) For linear classifiers in $\mathbb{R}^d$, $VCdim = d + 1$
 - (F) A higher VC dimension always leads to better generalization
 - (T) The generalization bound depends on the ratio n/ν
 - (T) To prove $VCdim(\mathcal{F}) = \nu$, we must show both that some set of size ν can be shattered and no set of size $\nu + 1$ can be shattered
- **Group 10: SVM Margin and Duality** In Support Vector Machines:
 - (T) The margin $\mu = 1/\|w\|$ for the hard-margin SVM
 - (T) The dual formulation involves maximizing $\sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y^i y^j (x^i \cdot x^j)$
 - (F) All training points have $\alpha_i > 0$ in the optimal solution
 - (T) The decision boundary is $f(x) = \text{sign}(\sum_i \alpha_i y^i k(x^i, x) + b)$
 - (T) Increasing C in soft-margin SVM increases the risk of overfitting
- **Group 11: Kernel Construction** Valid operations for constructing new kernels:
 - (T) The sum $k_1 + k_2$ of two kernels is a kernel
 - (T) The product $k_1 \cdot k_2$ of two kernels is a kernel
 - (F) The quotient k_1/k_2 of two kernels is always a kernel
 - (T) $\exp(\gamma k)$ is a kernel for any kernel k and $\gamma > 0$
 - (T) Pointwise limits of kernel sequences are kernels
- **Group 12: Hopfield Networks** Properties of Hopfield networks:
 - (T) The energy function $H(s)$ always decreases or stays constant with asynchronous updates
 - (T) Storage capacity with perfect recall is approximately $n/(2 \ln(n))$
 - (F) Spurious states never occur if the stored patterns are orthogonal
 - (T) If ξ is a stored pattern, $-\xi$ is also stable (reversed pattern)
 - (T) Setting $w_{ii} = 0$ helps reduce spurious states
- **Group 13: Delta Rule and Gradient Descent** For the Delta Rule (least squares training):
 - (T) The update rule is $\Delta w_j = -\eta \sum_{i=1}^n (y^i - w \cdot x^i) x_j^i$
 - (F) The learning rate η can be arbitrarily large without affecting convergence
 - (T) Convergence requires $\eta < 2/\lambda_{max}$ where λ_{max} is the largest eigenvalue of the auto-correlation matrix
 - (T) Online learning ($n_0 = 1$) converges if $\sum \eta(t) = \infty$ and $\sum \eta^2(t) < \infty$

- (F) Batch gradient descent always converges faster than stochastic gradient descent
- **Group 14: Graph Neural Networks** About Graph Neural Networks:
 - (T) GNNs must satisfy permutation invariance: $GNN(PX, PAP^T) = GNN(X, A)$
 - (T) Message passing aggregates information from neighboring nodes
 - (F) The adjacency matrix A must be normalized before use in all GNN architectures
 - (T) Node embeddings are updated as $h_i^{(\ell+1)} = \Phi(w_{self}h_i^{(\ell)} + w_{neigh} \sum_{j \in N(i)} h_j^{(\ell)} + b)$
 - (T) GNNs can perform predictions at node, edge, and graph levels
- **Group 15: Gaussian Generative Classifiers** For Gaussian-based generative classifiers:
 - (T) The Bayes classifier is $f^*(x) = \arg \max_j \{P(x|c_j)P(c_j)\}$
 - (T) If all classes share the same covariance Σ , decision boundaries are hyperplanes
 - (F) The discriminant function is always linear regardless of the covariance structure
 - (T) With $\Sigma = \sigma^2 I$ and equal priors, the classifier reduces to nearest mean
 - (T) Decision boundaries are quadratic when covariances differ across classes
- **Group 16: Kernel-Network Connection** Regarding the connection between kernels and neural networks:
 - (T) As $H \rightarrow \infty$, a 1-hidden-layer MLP converges to a Gaussian Process
 - (T) The ReLU activation corresponds to a specific kernel function
 - (F) Kernel methods and neural networks always produce identical decision boundaries
 - (T) The ℓ -fold composition of kernels relates to deep network architectures
 - (T) The arc-cosine kernel corresponds to neural networks with specific activations
- **Group 17: Cross-Validation and Model Selection** About model evaluation and selection:
 - (T) k -fold cross-validation splits data into k subsets for training and validation
 - (F) Test data should be used during hyperparameter tuning
 - (T) The generalization error is estimated as $R[\hat{f}] = \mathbb{E}[(Y - \hat{f}(X))^2]$
 - (T) Proper resampling helps avoid overfitting and methodological biases
 - (F) A single train-test split is sufficient for reliable performance estimation
- **Group 18: Regularization Techniques** Regarding regularization in machine learning:
 - (T) Ridge regression corresponds to MAP estimation with Gaussian priors on weights
 - (T) Regularization helps convert ill-posed problems into well-posed ones
 - (F) Increasing the regularization parameter λ always improves test performance
 - (T) The regularizer $\Omega(f)$ typically measures the magnitude or complexity of f
 - (T) L2 regularization ($\|w\|^2$) encourages small but non-zero weights
- **Group 19: Distance Metrics** About distance and similarity measures:
 - (T) The Jaccard similarity $J(A, B) = |A \cap B|/|A \cup B|$ is valid for set data
 - (T) The Mahalanobis distance corresponds to using $A = \Sigma^{-1}$ in the weighted metric
 - (F) Euclidean distance is always the best choice for all data types
 - (T) Clarke's distance $d(x, z) = |x - z|/(x + z)$ is a semantic distance
 - (T) A distance matrix D is Euclidean if $(\mathbb{1} - \mathbf{1}s^T)\Delta(\mathbb{1} - s\mathbf{1}^T)$ is psd
- **Group 20: Risk and Empirical Risk** Fundamental concepts about risk:
 - (T) The risk $R[f] = \mathbb{E}[\ell(Y, f(X))]$ is the expected loss over the data distribution
 - (T) The empirical risk $\hat{R}[f] = \frac{1}{n} \sum_{i=1}^n \ell(y^i, f(x^i))$ is computed on training data
 - (T) For fixed f , the expected empirical risk equals the true risk: $\mathbb{E}[\hat{R}[f]] = R[f]$
 - (F) Minimizing empirical risk always leads to the optimal predictor
 - (T) The Bayes risk $R[f^*]$ represents the lowest achievable error